

PHYS345: Advanced Methods of Experimental Physics

SPCE345: Advanced Experiments in Space Science

Question Bank

Use any online resources required - I certainly did!

Earthquake Data

Assignment question

The link <https://raw.githubusercontent.com/plotly/datasets/master/earthquakes-23k.csv> contains 50 years of earthquake data, including the magnitude (on the Richter scale) and the coordinates of the epicentre

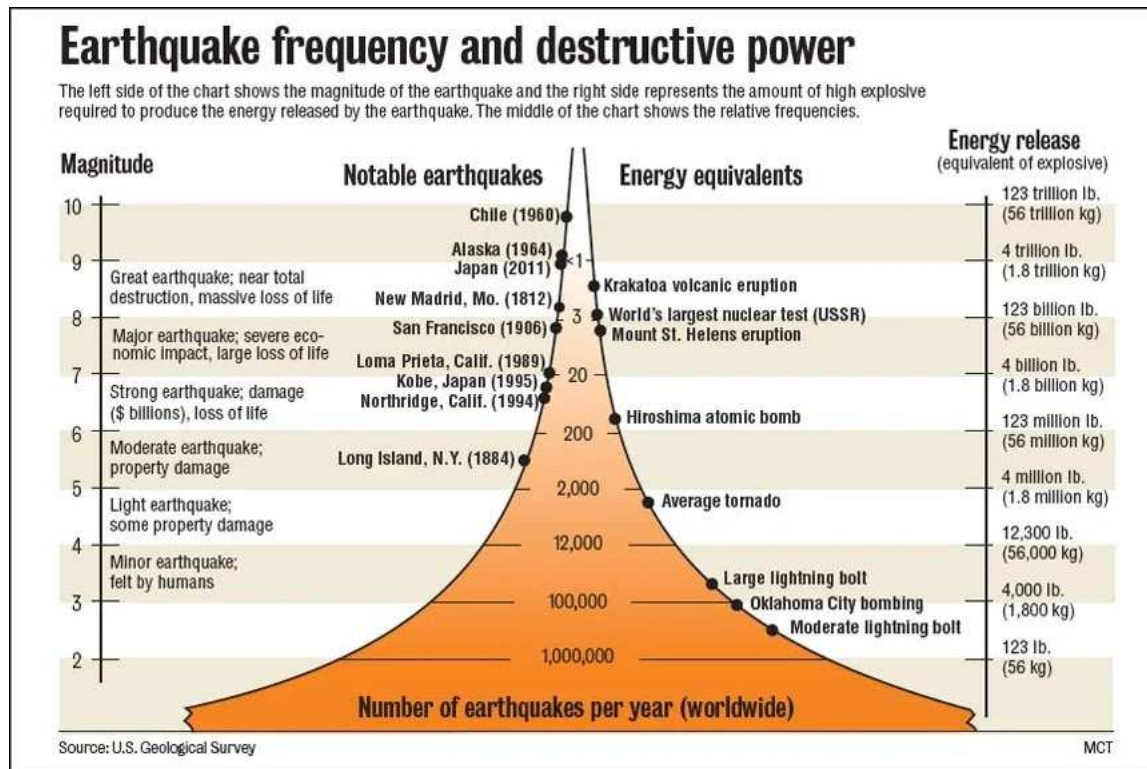
- (a) Looking at the data:
 - (i) What is the range of magnitudes?
 - (ii) What are the mean and median magnitudes?
 - (iii) At which location(s) was the strongest earthquake recorded?
- (b) Show the distribution of magnitudes in a histogram.
- (c) Produce bar plots of:
 - (i) The number of earthquakes grouped by the month name.
 - (ii) The mean magnitude grouped by the month name.

Hint: You will need to extract part of the date string. You may also need to convert a series to a dataframe, e.g. `df = series.to_frame()`
- (d) Plot the magnitudes versus the date.
- (e) You'll notice this took ages and is a bit of a mess, try to show this in a clearer format.
- (f) According to AI

"The Richter scale measures the magnitude of earthquakes, and each whole number increase on the scale represents a roughly 31.6 times increase in the energy released. A magnitude 6 earthquake, for example, releases about the same energy as a one-megaton nuclear explosion. The scale is logarithmic, meaning each whole number increase corresponds to a tenfold increase in the amplitude of the ground motion recorded by a seismograph."

Using the data in the plot below as a guide:

- (i) Estimate by how many factors the energy release spans between the weakest and strongest earthquakes recorded here.
- (ii) The largest earthquake ever recorded was the Valdivia earthquake in Chile on May 22, 1960, with a magnitude of 9.5. What was the equivalent energy in the same units (kg equivalent of explosive).



(g) Plot the latitude and longitude of the earthquakes using a third 'dimension' for the magnitudes (e.g. size/colour of marker or a heatmap).

If you are really motivated, you could try and superimpose this on a map of the world, as seen [here](#) or [here](#).

(h) And, not for the faint-hearted, if you are *really* motivated, use `countries.csv` to rank the world's countries by the average earthquake strength.

To do this you have to match the closest coordinates between the two datasets. Some methods are

- Brute force – comparing every set of coordinates in one dataset against the other and recording the distance between them. This involves $23409 \times 244 = 5\,711\,796$ comparisons so may take a while!
- Divide and conquer – where the points are split into smaller and smaller lists (e.g. [here](#))
- Balltree – the one I used, based on the code at the bottom of [here](#).¹ Dissecting their example

```
airports = [(10,10),(20,20),(30,30),(40,40)]
airports_rad = np.array([[radians(x[0]), radians(x[1])] for x in airports])
tree = BallTree(airports_rad, metric = 'haversine')
result = tree.query([radians(21),radians(21)])
```

Line 1 – `airports`, for us this would be the earthquake coordinates *in an array*

Line 2 – converting these coordinates to radians. I did this prior in both dataframes, e.g.

```
c['Longitude (average)'] = c['Longitude (average)'] * np.pi / 180
```

Line 4 – Comparing with the coordinate set (21,21), after conversion to radians. This will be the country coordinates. Note the result is two arrays, the first of which contains the distances and the second the indices of the original data

After this you would have to combine the results with the earthquake data.

¹Also [here](#)

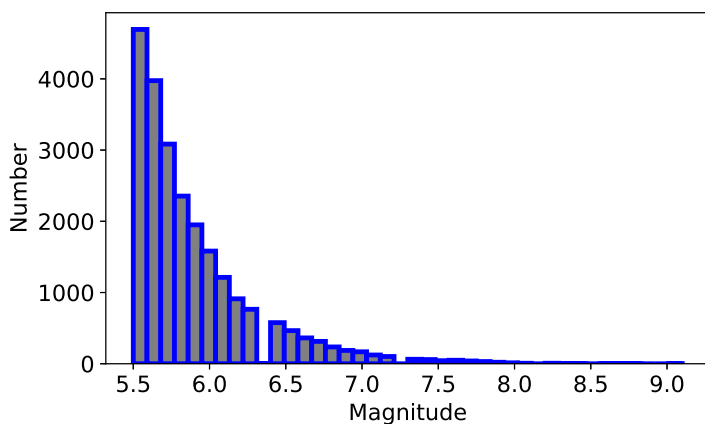
(a) Looking at the data:

```
(i) count  23412.000000  23412.000000  23412.000000
      mean    1.679033    39.639961    5.882531
      std     30.113183   125.511959    0.423066
      min     -77.080000  -179.997000    5.500000
      25%     -18.653000  -76.349750    5.600000
      50%     -3.568500   103.982000    5.700000
      75%     26.190750   145.026250    6.000000
      max      86.005000   179.998000    9.100000
```

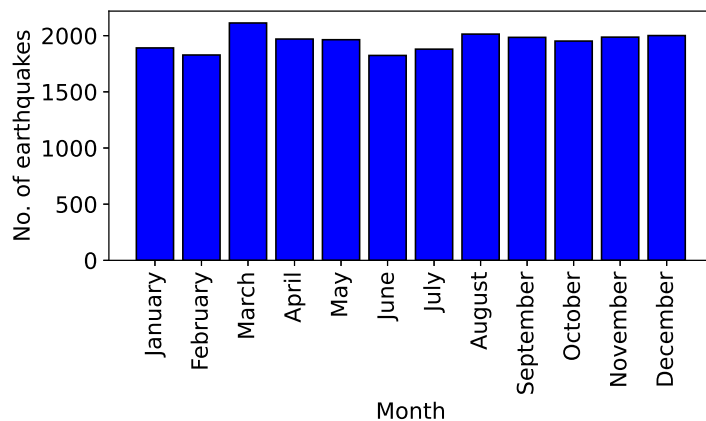
5.5 to 9.1

(ii) The mean is 5.88 and the median 5.70

```
(iii)           Date  Latitude  Longitude  Magnitude
      17083  12/26/2004    3.295    95.982        9.1  off the coast of Sumatra
      20501  03/11/2011   38.297   142.373        9.1  off the coat of Japan
```

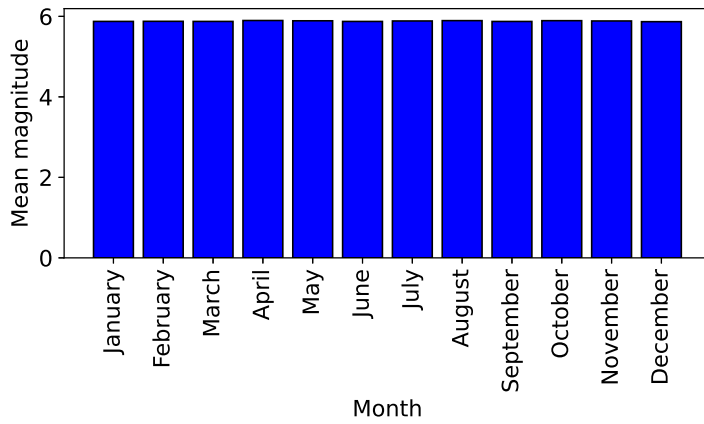


(b)



(c) (i)

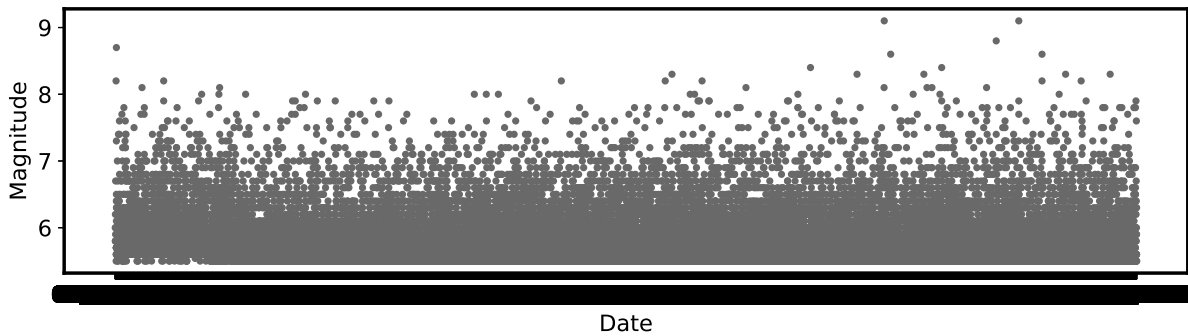
²Dirty, working version in quakes.py



(ii)

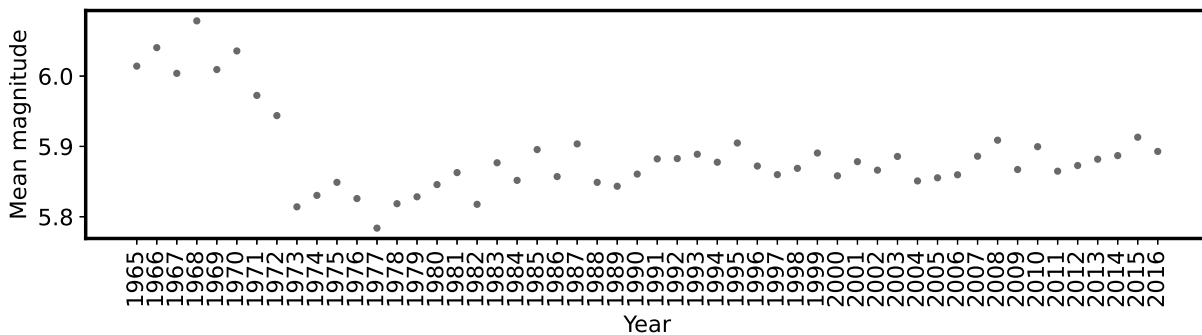
These all look the same, but just very similar

Magnitude			
month			
January	5.874849	July	5.885883
February	5.877188	August	5.894374
March	5.874633	September	5.873804
April	5.897411	October	5.893207
May	5.890214	November	5.885994
June	5.874271	December	5.868406



(d)

(e) One way would be to show the average magnitude grouped by year.



The late 1960s seem to have more seismic activity, but note that the data do not contain any magnitudes < 5.5 and so this is only an average of the strongest earthquakes, not all.

(f) (i) *the scale is logarithmic*, so something of the form

$$\underbrace{M - 5}_{\text{As } E = E_0 \text{ gives } 0} = C \log \left(\frac{E}{E_0} \right), \text{ where } E_0 = 1.8 \times 10^6 \text{ kg explosive}$$

E.g. $E = 1.8 \times 10^6 \times 31.6 = 56.9 \times 10^6 \text{ kg}$

$$1 = C \log(31.6) = 1.5C \Rightarrow C = 0.667$$

$$\Rightarrow M = 0.667 \log \left(\frac{E}{1.8 \times 10^6} \right) + 5$$

$$\Rightarrow E = 1.8 \times 10^6 \times 10^{1.5(M-5)},$$

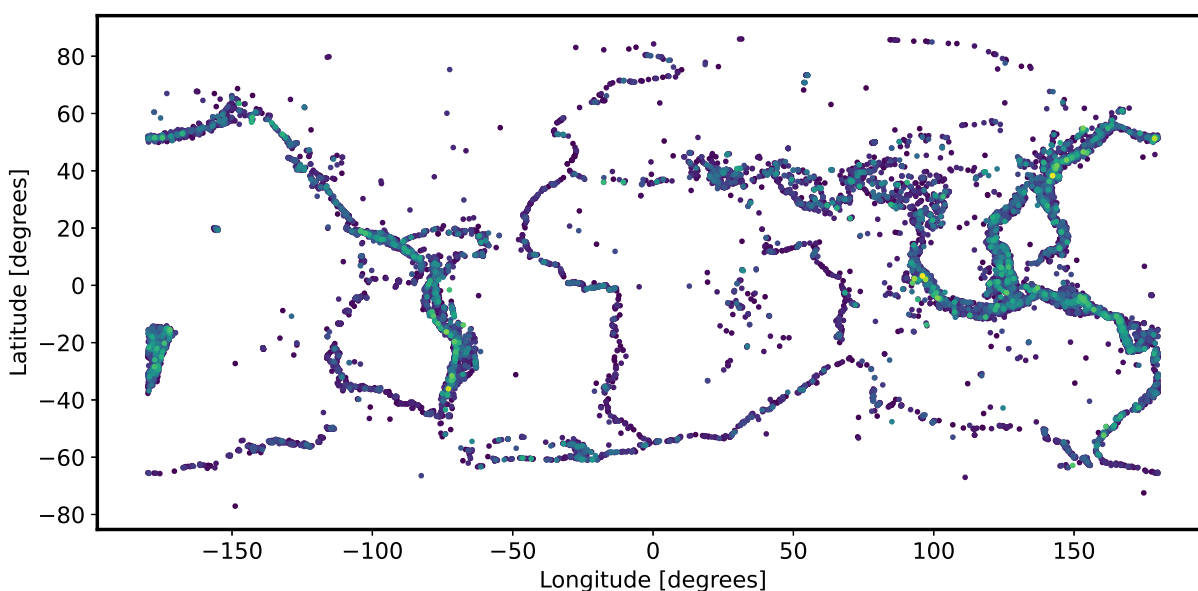
$$M = 5.5 \Rightarrow E = 1.8 \times 10^6 \times 10^{1.5(0.5)} = 1.0 \times 10^7 \text{ kg}$$

$$M = 9.1 \Rightarrow E = 1.8 \times 10^6 \times 10^{1.5(4.1)} = 2.5 \times 10^{12} \text{ kg}$$

A factor of 250 000.

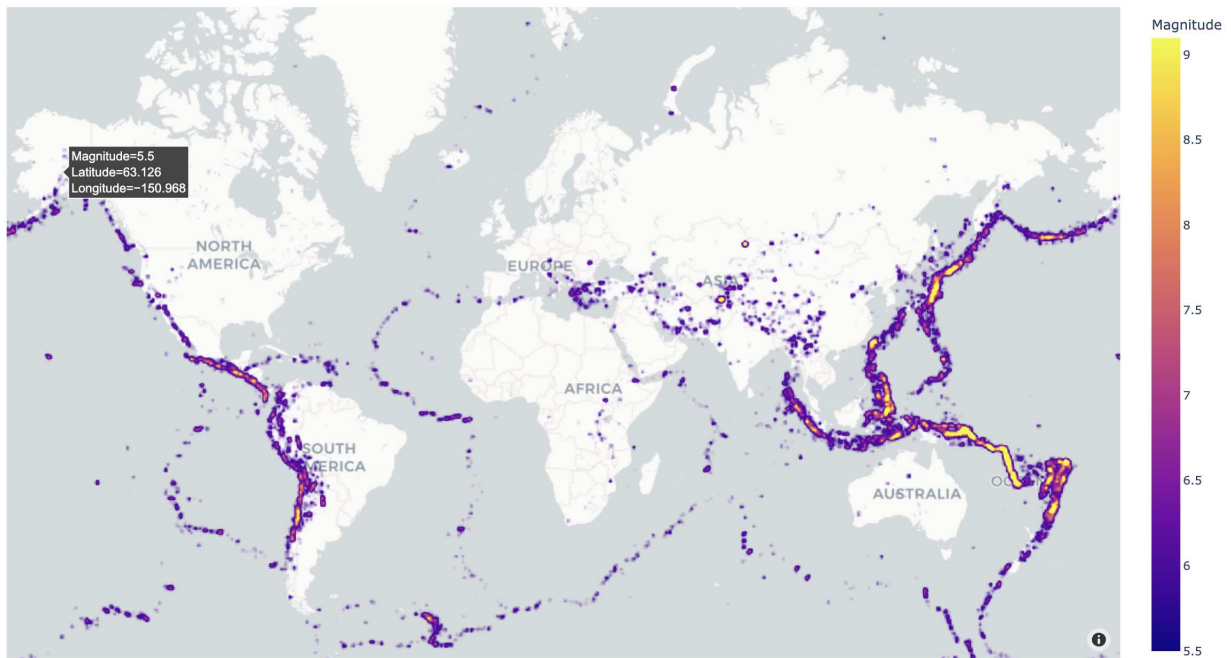
Happy if students interpolate instead or obtain a reasonable value by another method

(ii) $M = 9.5 \Rightarrow E = 1.8 \times 10^6 \times 10^{1.5(4.5)} = 1.0 \times 10^{13} \text{ kg}$

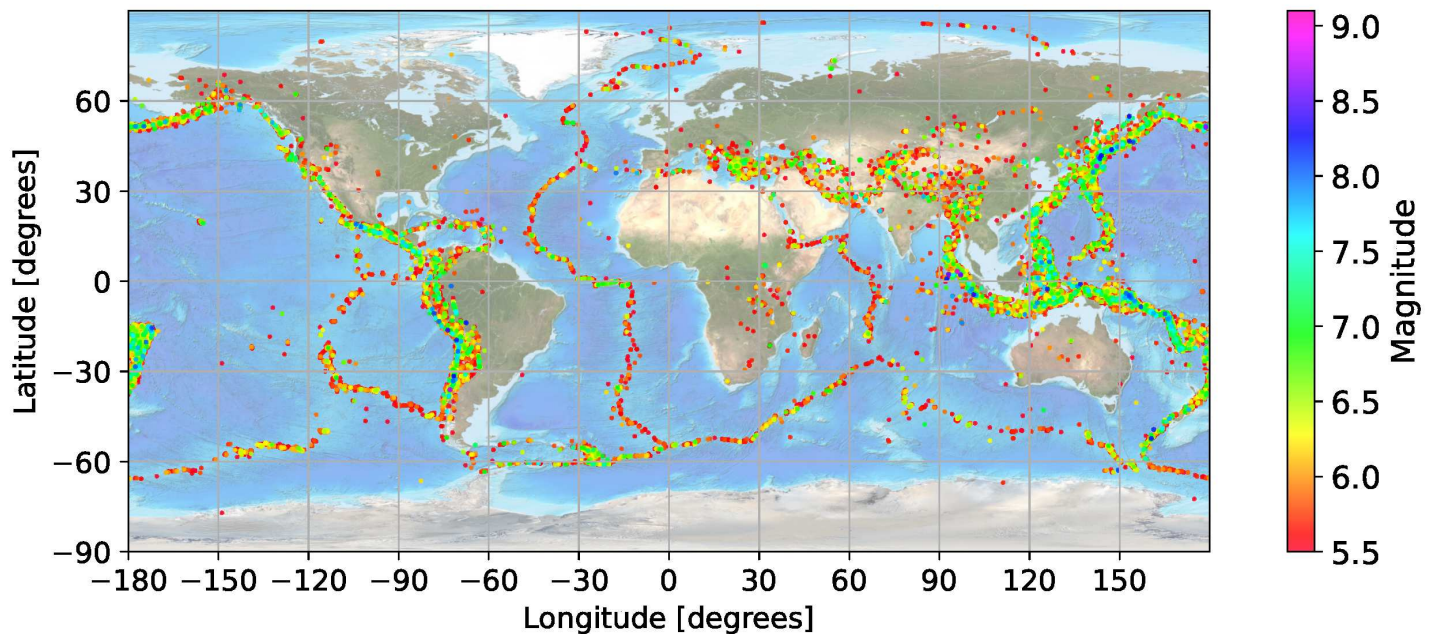


(g)

First option is pretty straightforward and opens the image on a browser (if run on command line). Is interactive, e.g. zooming. However the magnitude heatmap doesn't seem that accurate (hover and see).



Second option requires a bit more effort, but being in matplotlib there is more control



I don't want to alarm you, but you can't even see New Zealand for points!³

(h) The final combined dataframe should look something like

	Date	Latitude	Longitude	Magnitude	dist	Country	Latitude (average)	Longitude (average)
0	08/13/1965	-15.9950	166.9600	6.1	0.000677	Vanuatu	-16.0000	167.0000
2	02/05/1966	39.0930	21.7770	6.2	0.003431	Greece	39.0000	22.0000
3	02/07/1966	29.9560	69.7610	6.5	0.003694	Pakistan	30.0000	70.0000
..	...	...	...	...	...	...	...	...
241	04/20/2016	13.2000	-55.8867	5.6	0.168103	French Guiana	4.0000	-53.0000
242	09/03/2016	36.4251	-96.9291	5.8	0.027505	United States	38.0000	-97.0000
243	09/22/2016	-21.5525	33.4112	5.6	0.061907	Zimbabwe	-20.0000	30.0000

³I've also lived in Chile!

and grouping these by country

Magnitude	Country
7.8	Nepal
7.2	Romania
7.0	Georgia
7.0	Turkmenistan
6.9	Montenegro
...	...
5.5	Grenada
5.5	Sudan
5.5	Germany
5.5	French Southern Territories
5.5	Afghanistan

Note that New Zealand is quite far down the list with a mean magnitude of 5.8. Maybe worth looking at which has the most earthquakes?

The file 0108+388_HI.dat is a spectrum towards the radio source COINS J0111+3906 (B0108+388) with the [Green Bank Telescope](#).



The units are frequency [MHz] and flux density [Jy] and the absorption due to the 21-cm transition of neutral hydrogen (H I) is located between $\nu = 849$ and 854 MHz.

- (a) Plot the spectrum over this range.
- (b) Frequency is not the most useful unit as it is redshift dependent. Velocity is not and offers more insight into the physics.

The optical redshift of the galaxy where the absorption arises is $z = 0.66847$.

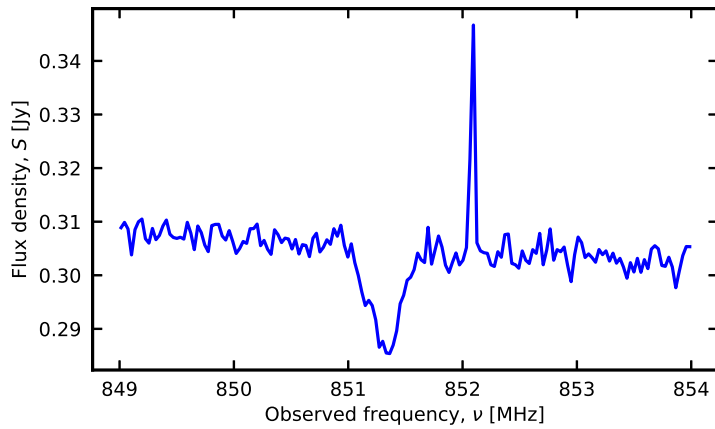
Given that the rest frequency of the H I 21-cm transition is $\nu_{\text{rest}} = 1420.40575$ MHz, using the relations

$$\nu_0 = \frac{\nu_{\text{rest}}}{z + 1} \text{ and } \Delta v \approx c \frac{\nu_0 - \nu}{\nu_0}, \text{ where } c \text{ is the speed of light.}$$

- (i) Convert the frequency into the velocity offset, Δv in km s^{-1} .
- (ii) Plot the spectrum with Δv as the x -axis.
- (iii) Comment on why you think the spectrum has 'flipped'.
- (c) Next, we want to get rid of that nasty spike (due to radio frequency interference) at $\Delta v \approx 250 \text{ km s}^{-1}$.
 - (i) At which velocity does this peak?
 - (ii) Remove and/or replace this and re-plot the spectrum.
- (d) There still remains a slope in the bandpass (the part of the spectrum not containing the absorption). Using your best judgement of where the absorption begins and ends:

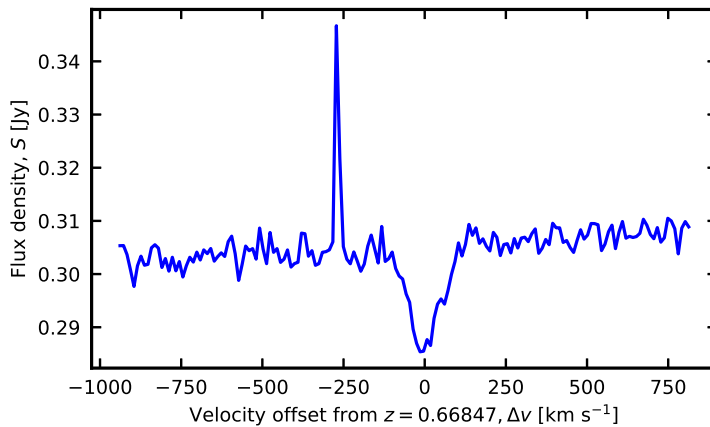
- (i) Fit a first order polynomial (linear fit) to the absorption free channels.
- (ii) Remove this from the spectrum.
- (iii) What is the peak flux of the absorption profile in relation to the bandpass?
- (iv) What is the rms noise level of the absorption free channels?

(a) Don't expect plots to be as polished as mine



(b) (i)

	index	MHz	Jy	v
0	732	853.98699	0.305336	-938.361624
1	733	853.95648	0.305358	-927.617552
2	734	853.92596	0.303664	-916.869959
..	...	...	...	...
161	893	849.07397	0.308638	791.754349
162	894	849.04346	0.309870	802.498420
163	895	849.01294	0.308875	813.246013

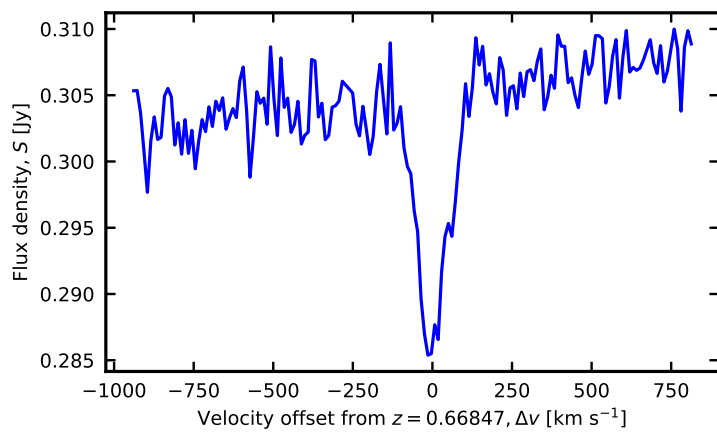
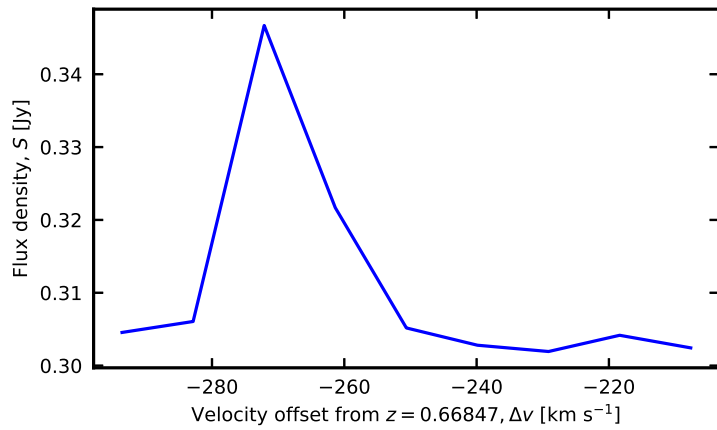


(ii)

(iii) The lower the frequency, the higher the redshift and, hence, the higher the velocity. The frequencies above that of the absorption are in fact blueshifted in relation to it and so the velocities are negative.

(c) (i) The peak of 0.35 Jy occurs at $v = -272.11$ km/s

⁴Dirty, working version in spectrum.py

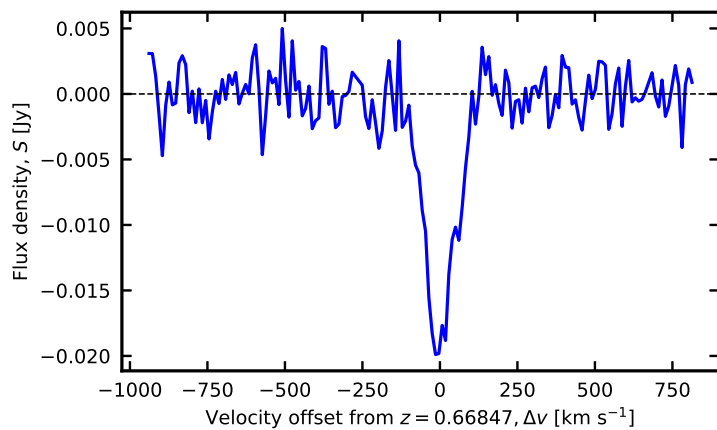


(ii)

(d) From the cursor estimate limits at -98 and 103 km s^{-1} .

(i) I get $m = 3.281 \times 10^{-6}$ and $c = 0.3053$.

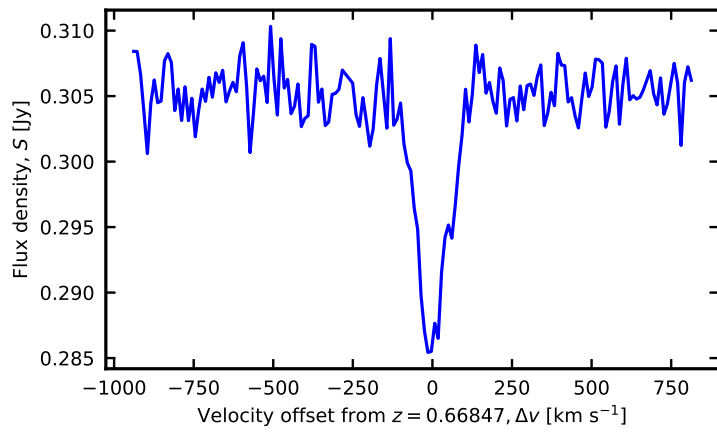
(ii)	index	MHz	Jy	v	bl	flux
0	732	853.98699	0.305336	-938.361624	0.302247	0.003089
1	733	853.95648	0.305358	-927.617552	0.302282	0.003076
..	...	...	...	...	...	...
162	894	849.04346	0.309870	802.498420	0.307959	0.001912
163	895	849.01294	0.308875	813.246013	0.307994	0.000881



If feeling fancy can add the mean flux ($c = 0.3053$) back in, but not necessary.

index	MHz	Jy	v	bl	flux	S
-------	-----	----	---	----	------	---

0	732	853.98699	0.305336	-938.361624	0.302247	0.003089	0.308415
1	733	853.95648	0.305358	-927.617552	0.302282	0.003076	0.308402
..	...	...	...	...	...	...	...
162	894	849.04346	0.309870	802.498420	0.307959	0.001912	0.307237
163	895	849.01294	0.308875	813.246013	0.307994	0.000881	0.306207



(iii) Peak flux is -0.020 Jy at -35.7 km/s

(iv) 0.0045 Jy